使用Astra Control Service

Astra Control Service

NetApp
May 25, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/astra-control-service/use/log-in.html on May 25, 2022. Always check docs.netapp.com for the latest.

目錄

使用Astra Control Service 1
登入Astra Control Service 1
管理及保護應用程式 1
檢視應用程式與運算健全狀況 26
管理儲存庫 28
管理您的帳戶 30
取消管理應用程式和叢集 38

使用Astra Control Service

登入Astra Control Service

若要存取Astra Control Service、請前往：透過SaaS型使用者介面 https://astra.netapp.io。

> 您可以使用單一登入、從公司目錄（聯盟身分識別）使用認證登入。若要深入瞭解、請前往 "Cloud Central說明中心" 然後選取* Cloud Central登入選項*。

您需要的是 **#8217** ；需要的是什麼

- "Cloud Central使用者ID"。
- "新的Astra Control帳戶" 或 "邀請現有帳戶"。
- 支援的網頁瀏覽器。

  Astra Control Service支援最新版本的Firefox、Safari和Chrome、最低解析度為1280 x 720。

步驟

1. 開啟網頁瀏覽器並前往 https://astra.netapp.io。
2. 使用您的 NetApp Cloud Central 認證登入。

管理及保護應用程式

開始管理應用程式

您先請 "將Kubernetes叢集新增至Astra Control"、您可以在叢集上安裝應用程式（Astra Control之外）、然後前往Astra Control的「應用程式」頁面、開始管理應用程式。

應用程式管理需求

請考慮下列Astra Control應用程式管理需求：

- 授權：若要使用Astra Control Center管理應用程式、您需要Astra Control Center授權。
- 命名空間：Astra Control要求應用程式不超過一個命名空間、但命名空間可以包含多個應用程式。
- * StorageClass *：如果您安裝的應用程式已明確設定StorageClass、且需要複製應用程式、則複製作業的目標叢集必須具有原本指定的StorageClass。將具有明確設定StorageClass的應用程式複製到沒有相同StorageClass的叢集、將會失敗。
- * Kubernetes資源*：使用未由Astra Control收集的Kubernetes資源的應用程式、可能沒有完整的應用程式資料管理功能。Astra Control會收集下列Kubernetes資源：

叢集角色	ClusterRoleBinding	組態對應
CustomResourceDesDefinition	CustomResource	可關係工作
示範	HorizontalPodAutoscaler	入侵

部署組態	互鎖Webhook	PeristentVolume Claim
Pod	Podcast中斷預算	Podcast範本
網路原則	ReplicaSet	角色
角色繫結	路由	秘密
服務	服務帳戶	狀態集
驗證Webhook		

支援的應用程式安裝方法

您可以使用下列方法安裝想要使用Astra Control管理的應用程式：

- 資訊清單檔案：Astra Control支援使用KUbectl從資訊清單檔案安裝的應用程式。例如：

```
kubectl apply -f myapp.yaml
```

- * Helm 3*：如果您使用Helm來安裝應用程式、Astra Control需要Helm版本3。完全支援使用Helm 3（或從Helm 2升級至Helm 3）來管理及複製應用程式。不支援管理以Helm 2安裝的應用程式。
- 操作員部署的應用程式：Astra Control支援以命名空間範圍運算子安裝的應用程式。這些運算子通常採用「傳遞值」而非「傳遞參照」架構來設計。以下是一些遵循這些模式的營運者應用程式：
  - "Apache K8ssandra"

    > 對於k8ssandra、支援就地還原作業。若要還原新命名空間或叢集的作業、必須先關閉應用程式的原始執行個體。這是為了確保傳遞的對等群組資訊不會導致跨執行個體通訊。不支援複製應用程式。

  - "Jenkins CI"
  - "Percona XtraDB叢集"

請注意、Astra Control可能無法複製以「傳遞參考」架構設計的操作員（例如、CockroachDB操作員）。在這些類型的複製作業中、複製的操作員會嘗試從來源操作員參考Kubernetes機密、儘管在複製程序中有自己的新秘密。由於Astra Control不知道來源營運者的Kubernetes機密資料、因此複製作業可能會失敗。

> 運算子及其安裝的應用程式必須使用相同的命名空間；您可能需要修改運算子的部署.yaml檔案、以確保情況如此。

在叢集上安裝應用程式

現在您已將叢集新增至Astra Control、即可在叢集上安裝應用程式。在預設情況下、會在新的儲存類別上配置持續磁碟區。在Pod上線後、您可以使用Astra Control來管理應用程式。

Astra Control只有在儲存設備位於由Astra Control安裝的儲存類別時、才會管理狀態化應用程式。

- "深入瞭解GKE叢集的儲存課程"
- "深入瞭解適用於高效能叢集的儲存類別"

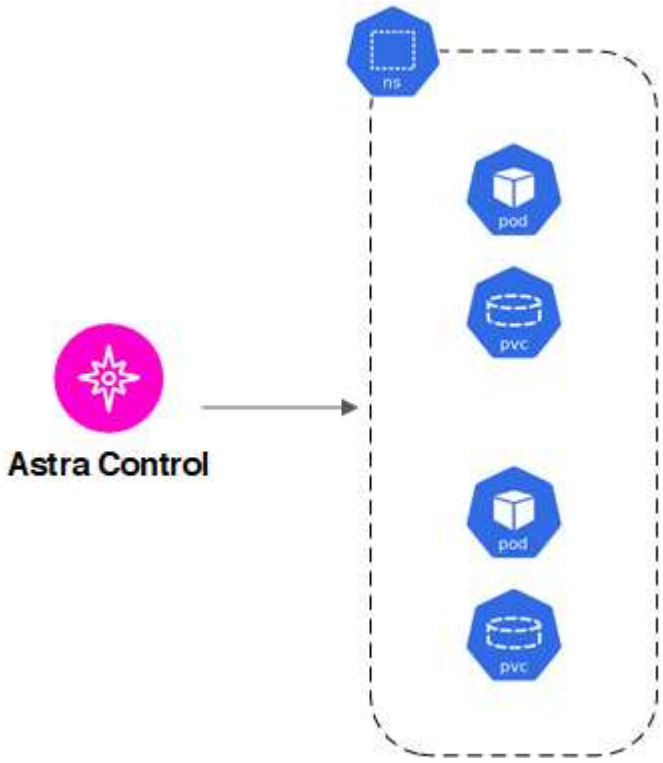
如需從Helm圖表部署一般應用程式的說明、請參閱下列內容：

- "從Helm圖表部署MariaDB"
- "從Helm圖表部署MySQL"
- "從Helm圖表部署Postgres"
- "從Helm圖表部署Jenkins"

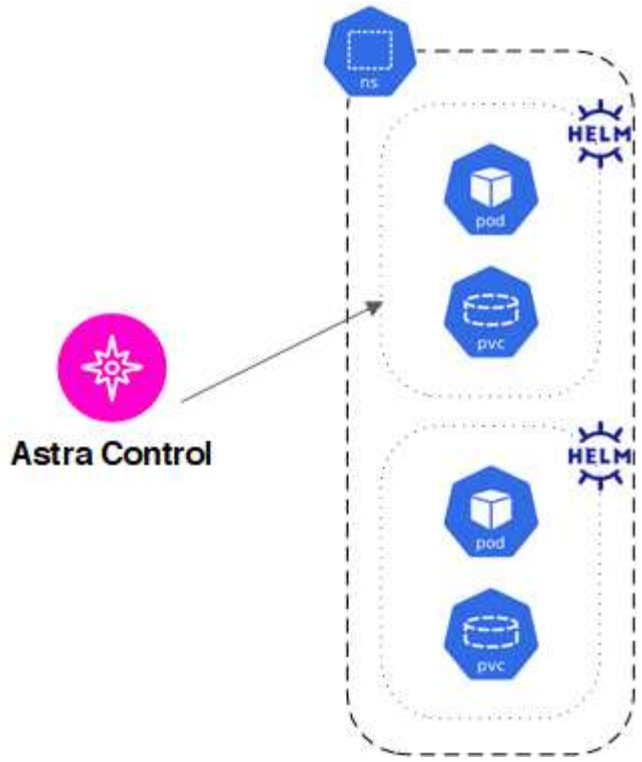
管理應用程式

當Astra Control探索叢集上執行的應用程式時、這些應用程式會處於不受管理的狀態、直到您選擇要管理的方式為止。Astra Control中的託管應用程式可以是下列任一項：

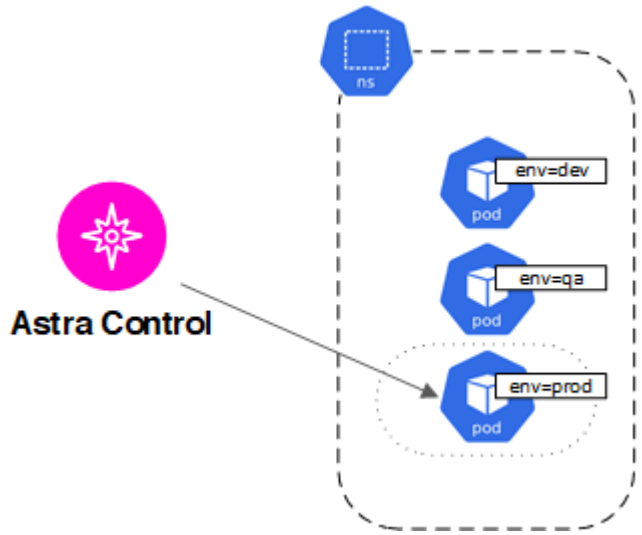
- 命名空間、包括該命名空間中的所有資源



- 在命名空間內部署有helm3的個別應用程式



- 一組資源、由Kubernetes標籤（在Astra Control中稱為_custom app_）在命名空間內識別



以下各節將說明如何使用這些選項來管理應用程式。

依命名空間管理應用程式

「應用程式」頁面的「探索到」區段會顯示這些命名空間中的命名空間、以及已安裝Helm的應用程式或自訂標記的應用程式。您可以選擇個別或在命名空間層級管理每個應用程式。所有這些都達到資料保護作業所需的精細度。

例如、您可能想要設定每週執行時間的「MARIA」備份原則、但您可能需要比這更頻繁地備份「MariaDB」（位於同一個命名空間）。根據這些需求、您需要分別管理應用程式、而非在單一命名空間下管理。

雖然Astra Control可讓您分別管理階層的兩個層級（命名空間和命名空間中的應用程式）、但最佳實務做法是選

擇一個或另一個層級。如果在命名空間和應用程式層級同時執行動作、則Astra Control中所採取的動作可能會失敗。

步驟

1. 選取*應用程式*、然後選取*已探索*。
2. 檢視探索到的命名空間清單、然後展開命名空間以檢視應用程式和相關資源。

   Astra Control會在命名空間中顯示Helm應用程式和自訂標記的應用程式。如果有Helm標籤、則會以標籤圖示標示。

3. 決定要個別管理每個應用程式、還是在命名空間層級管理。
4. 在階層中所需的層級、選取* Actions （動作）欄中的下拉式清單、然後選取* Manage （管理）。
5. 如果您不想管理應用程式、請在所需應用程式的*「Actions」（動作）欄中選取下拉式清單、然後選取「Ignori*」（忽略）。

   例如、如果您想要一起管理「Jenkins」命名空間下的所有應用程式、使其具有相同的快照和備份原則、您可以管理命名空間並忽略命名空間中的應用程式。

您選擇管理的應用程式現在可從*「託管」*索引標籤取得。任何忽略的應用程式都會移至*忽略*索引標籤。理想情況下、探索到的索引標籤會顯示零應用程式、以便在安裝新應用程式時、更容易找到及管理。

依**Kubernetes**標籤管理應用程式

Astra Control在應用程式頁面頂端包含一個名為*定義自訂應用程式*的動作。您可以使用此動作來管理以Kubernetes標籤識別的應用程式。 "深入瞭解如何透過Kubernetes標籤定義應用程式"。

步驟

1. 選擇*應用程式>定義自訂應用程式*。
2. 在「定義自訂應用程式」對話方塊中、提供管理應用程式所需的資訊：
   a. 新應用程式：輸入應用程式的顯示名稱。
   b. 叢集：選取應用程式所在的叢集。
   c. 命名空間：選取應用程式的命名空間。
   d. 標籤：輸入標籤或從下列資源中選取標籤。
   e. 選取的資源：檢視及管理您要保護的選定Kubernetes資源（Pod、機密、持續磁碟區等）。
      - 展開資源並選取標籤數量、即可檢視可用的標籤。
      - 選取其中一個標籤。

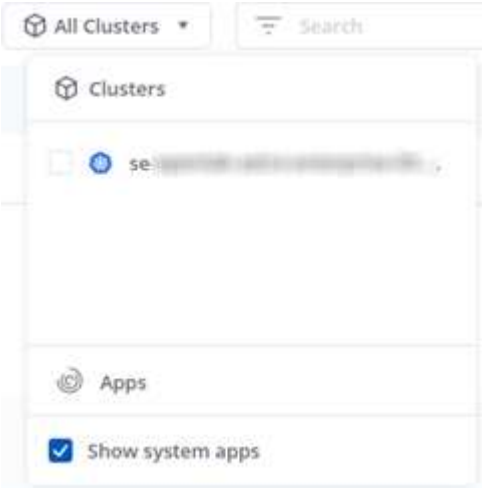
        選擇標籤後、標籤會顯示在*標籤*欄位中。Astra Control也會更新*未選取的資源*區段、以顯示與所選標籤不符的資源。

   f. 未選取的資源：確認您不想保護的應用程式資源。
3. 選擇*定義自訂應用程式*。

Astra Control可管理應用程式。您現在可以在*託管*索引標籤中找到它。

系統應用程式呢？

Astra Control也會探索Kubernetes叢集上執行的系統應用程式。您可以篩選應用程式清單來檢視這些項目。



我們預設不會顯示這些系統應用程式、因為您很少需要備份這些應用程式。

利用快照與備份來保護應用程式

使用自動保護原則或臨機操作、拍攝快照和備份資料、保護您的應用程式。

快照與備份

_snapshot是應用程式的時間點複本、儲存在與應用程式相同的已配置磁碟區上。通常速度很快。本機快照可用來將應用程式還原至較早的時間點。

備份_儲存在雲端的物件儲存設備上。相較於本機快照、備份速度可能較慢。但可在雲端跨區域存取、以進行應用程式移轉。您也可以選擇較長的備份保留期間。

> 您必須等到最近進行備份之後、才能獲得完整保護。這很重要、因為備份儲存在遠離持續磁碟區的物件存放區中。如果故障或意外損壞叢集及其持續儲存設備、則需要備份才能恢復。快照無法讓您恢復。

設定保護原則

保護原則可在已定義的排程中建立快照、備份或兩者、以保護應用程式。您可以選擇每小時、每天、每週和每月建立快照和備份、也可以指定要保留的複本數量。如果您需要每小時執行一次以上的備份或快照、您可以 "使用Astra Control REST API建立快照與備份"。

步驟

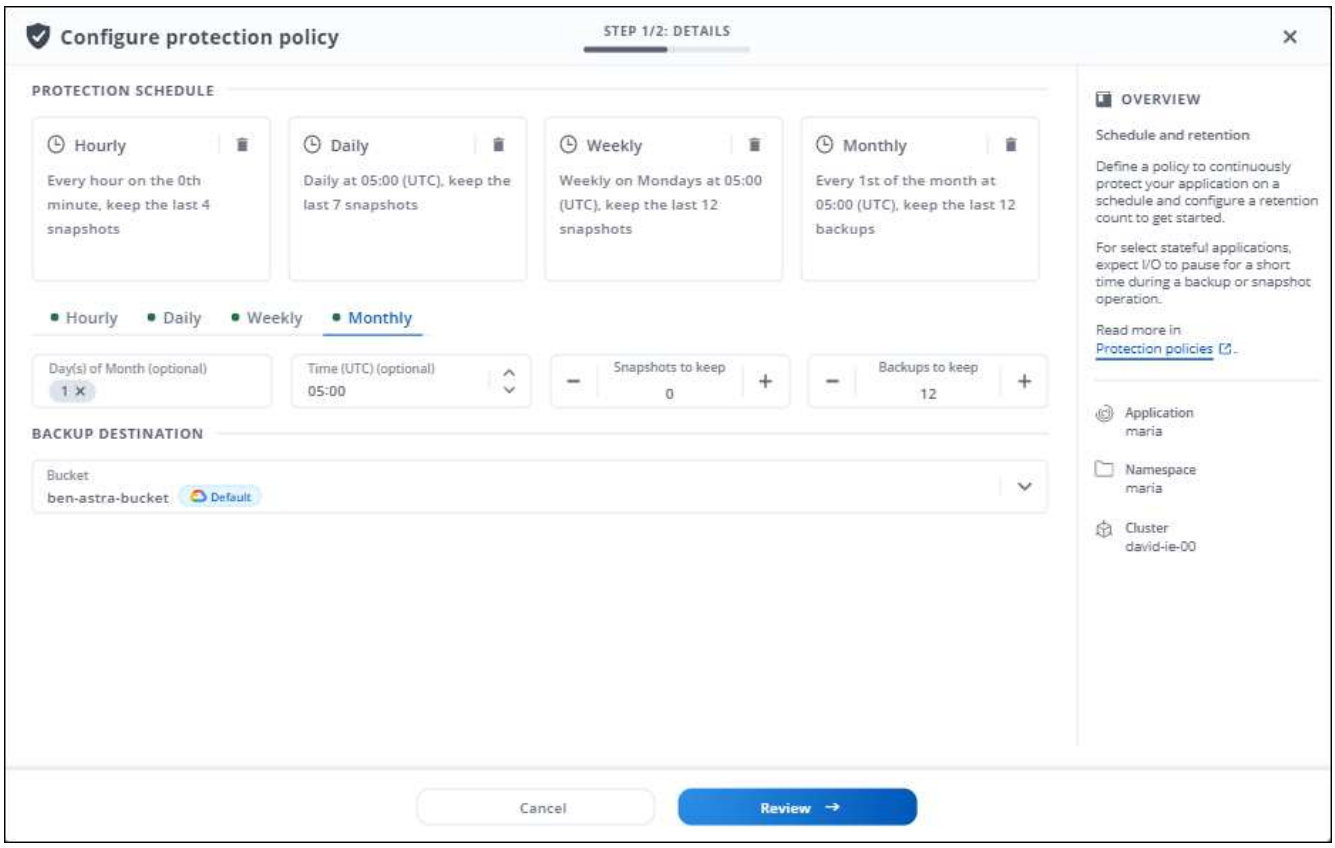
1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。
3. 選取*設定保護原則*。
4. 選擇每小時、每日、每週和每月排程所需保留的快照和備份數量、以定義保護排程。

   您可以同時定義每小時、每日、每週及每月排程。在您設定快照和備份的保留層級之前、排程不會變成作用

中。

當您設定備份的保留層級時、可以選擇要儲存備份的儲存區。

下列範例設定四種保護排程：每小時、每日、每週及每月提供快照與備份。



5. 選擇* Review *。
6. 選擇*設定*。

以下是每個步驟的影片。

► https://docs.netapp.com/zh-tw/astra-control-service/media/use/video-set-protection-policy.mp4 *(video)*

Astra Control會使用您定義的排程和保留原則來建立和保留快照和備份、以實作資料保護原則。

建立快照

您可以隨時建立隨需快照。

步驟

1. 選擇*應用程式*。
2. 選取所需應用程式*「Actions」（動作）*欄中的下拉式清單。
3. 選擇* Snapshot *。
4. 自訂快照名稱、然後選取*檢閱資訊*。
5. 檢閱快照摘要、然後選取* Snapshot App*。

Astra Control會建立應用程式的快照。

建立備份

您也可以隨時備份應用程式。

步驟

1. 選擇*應用程式*。
2. 選取所需應用程式*「Actions」（動作）*欄中的下拉式清單。
3. 選擇*備份*。
4. 自訂備份名稱、選擇是否要從現有的快照備份應用程式、然後選取*檢閱資訊*。
5. 檢閱備份摘要、然後選取*備份應用程式*。

Astra Control會建立應用程式的備份。

檢視快照與備份

您可以從「資料保護」索引標籤檢視應用程式的快照與備份。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。

   快照預設會顯示。

3. 選取*備份*以查看備份清單。

刪除快照

刪除不再需要的排程或隨需快照。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。
3. 在* Actions（操作）*列中選擇所需快照的下拉列表。
4. 選擇*刪除snapshot *。
5. 輸入快照名稱以確認刪除、然後選取*是、刪除快照*。

Astra Control會刪除快照。

刪除備份

刪除不再需要的排程或隨需備份。

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。

3. 選擇*備份*。
4. 在* Actions（操作）*列中選擇下拉列表以進行所需的備份。
5. 選擇*刪除備份*。
6. 輸入備份名稱以確認刪除、然後選擇*是、刪除備份*。

Astra Control會刪除備份。

還原應用程式

Astra Control可以從快照或備份還原應用程式。將應用程式還原至同一個叢集時、從現有的快照還原速度會更快。您可以使用Astra Control UI或 "Astra Control API" 以還原應用程式。

> 任何具有命名空間名稱/ ID或命名空間標籤限制的成員使用者、都可以將應用程式複製或還原到同一個叢集上的新命名空間、或是組織帳戶中的任何其他叢集。不過、相同的使用者無法存取新命名空間中的複製或還原應用程式。在複製或還原作業建立新命名空間之後、帳戶管理員/擁有者可以編輯成員使用者帳戶、並更新受影響使用者的角色限制、以便授予新命名空間的存取權。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。
3. 如果您想要從快照還原、請保持選取* Snapshot*。否則、請選取*備份*以從備份還原。
4. 在* Actions （操作）欄中選取您要還原的快照或備份下拉式清單。
5. 選取*還原應用程式*。
6. 還原詳細資料：指定還原應用程式的詳細資料。若要就地還原應用程式（將應用程式還原為本身的舊版）、請選擇目前執行的相同命名空間和目的地叢集：
   - 輸入應用程式的命名空間。
   - 選擇應用程式的目的地叢集。
   - 按一下* Review *。
7. 還原摘要：檢閱還原動作的詳細資料、輸入「還原」、然後選取*還原*。

Astra Control會根據您提供的資訊還原應用程式。如果您就地還原應用程式、則會刪除並重新建立相關的持續磁碟區。

複製及移轉應用程式

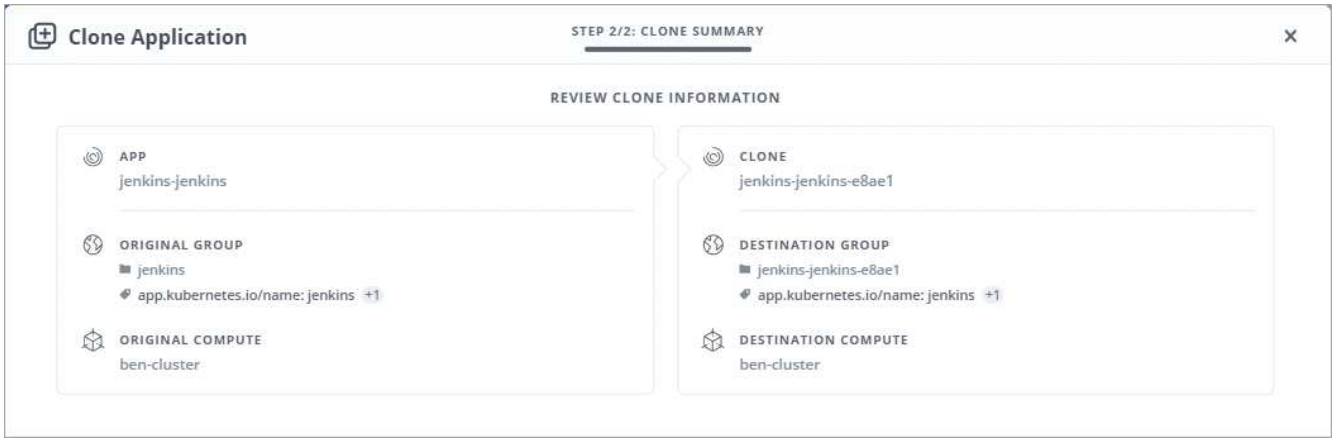
複製現有的應用程式、在相同的Kubernetes叢集或其他叢集上建立複製的應用程式。如果您需要將應用程式和儲存設備從一個Kubernetes叢集移至另一個叢集、複製作業將有助於您。例如、您可能想要透過CI/CD傳輸途徑和Kubernetes命名空間來移動工作負載。

當Astra Control複製應用程式時、會建立應用程式組態和持續儲存的複本。

- 如果您複製由操作人員部署的Jenkins CI執行個體、則需要手動還原持續性資料。這是應用程式部署模式的限制。
- 如果您部署的應用程式已明確設定StorageClass、且需要複製應用程式、則目標叢集必須具有原本指定的StorageClass。將具有明確設定StorageClass的應用程式複製到沒有相同StorageClass的叢集、將會失敗。
- 在應用程式備份或應用程式還原期間、您可以選擇性地指定庫位ID。不過、應用程式複製作業一律會使用已定義的預設儲存區。沒有選項可變更實體複本的儲存區。如果您想要控制所使用的儲存桶、您也可以選擇 "變更庫位預設值" 或執行 "備份" 接著是A "還原" 獨立提供。
- 任何具有命名空間名稱/ ID或命名空間標籤限制的成員使用者、都可以將應用程式複製或還原到同一個叢集上的新命名空間、或是組織帳戶中的任何其他叢集。不過、相同的使用者無法存取新命名空間中的複製或還原應用程式。在複製或還原作業建立新命名空間之後、帳戶管理員/擁有者可以編輯成員使用者帳戶、並更新受影響使用者的角色限制、以便授予新命名空間的存取權。

步驟

1. 選擇*應用程式*。
2. 選取所需應用程式*「Action"（行動）」欄中的下拉式清單。
3. 選擇* Clone（克隆）*。
4. 複製詳細資料：指定複製的詳細資料：
   - 保留預設名稱和命名空間、或加以編輯。
   - 選擇要複製的目的地叢集。
   - 選擇是要從現有的快照或備份建立複本。如果您未選取此選項、Astra Control會從應用程式的目前狀態建立複本。
5. * Clone Summary（複製摘要）：檢閱有關複製的詳細資料、然後選取 Clone App*（複製應用程式*）。



Astra Control會根據您提供的資訊來複製該應用程式。

管理應用程式執行掛勾

執行攔截是自訂指令碼、您可以在託管應用程式的快照之前或之後執行。例如、如果您有資料庫應用程式、您可以使用執行掛勾來暫停快照之前的所有資料庫交易、並在快照完成後繼續交易。如此可確保應用程式一致的快照。

預設執行掛勾和規則運算式

對於某些應用程式、Astra Control隨附NetApp提供的預設執行掛勾、可處理快照前後的凍結和解凍作業。Astra Control使用規則運算式、將應用程式的容器映像與下列應用程式配對：

- MariaDB
  - 符合規則運算式：\bmariadb\b
- MySQL
  - 相符的規則運算式：\bmysql\b
- PostgreSQL
  - 相符的規則運算式：\bpostgresql\b

如果有相符項目、則NetApp提供的該應用程式預設執行掛勾會顯示在應用程式的作用中執行掛勾清單中、而這些掛勾會在擷取該應用程式的快照時自動執行。如果其中一個自訂應用程式有類似的映像名稱、正好符合其中一個規則運算式（而且您不想使用預設的執行掛勾）、您可以變更映像名稱、 或停用該應用程式的預設執行掛勾、改用自訂掛勾。

您無法刪除或修改預設的執行掛勾。

關於自訂執行掛勾的重要注意事項

規劃應用程式的執行掛勾時、請考量下列事項。

- Astra Control需要以執行Shell指令碼的格式寫入執行掛勾。
- 指令碼大小限制為128 KB。
- Astra Control會使用執行掛勾設定和任何符合條件來判斷哪些掛勾適用於快照。
- 所有執行掛機故障都是軟性故障、即使掛機故障、仍會嘗試其他掛機和快照。但是、當掛機失敗時、會在*活動*頁面事件記錄中記錄警告事件。
- 若要建立、編輯或刪除執行掛勾、您必須是擁有擁有者、管理員或成員權限的使用者。
- 如果執行掛機執行時間超過25分鐘、掛機將會失敗、並建立傳回代碼為「N/A」的事件記錄項目。任何受影響的快照都會逾時並標示為故障、並會出現一個事件記錄項目、指出逾時時間。

> 由於執行掛勾通常會減少或完全停用執行中應用程式的功能、因此您應該一律盡量縮短自訂執行掛勾執行所需的時間。

執行快照時、執行掛機事件會依照下列順序進行：

1. 任何適用的NetApp提供的預設快照前執行掛勾、都會在適當的容器上執行。
2. 任何適用的自訂快照前執行掛勾都會在適當的容器上執行。您可以視需要建立及執行任意數量的自訂快照前置掛勾、但在快照之前執行這些掛勾的順序並不保證也無法設定。
3. 快照即會執行。
4. 任何適用的自訂快照後執行掛勾都會在適當的容器上執行。您可以視需要建立及執行任意數量的自訂快照後掛勾、但快照後執行這些掛勾的順序並不保證也無法設定。
5. 任何適用的NetApp提供的預設快照後執行掛勾、都會在適當的容器上執行。

> 在正式作業環境中啟用執行攔截指令碼之前、請務必先進行測試。您可以使用'kubecl exec'命令來方便地測試指令碼。在正式作業環境中啟用執行掛勾之後、請測試所產生的快照、以確保它們一致。您可以將應用程式複製到暫用命名空間、還原快照、然後測試應用程式、藉此完成此作業。

檢視現有的執行掛勾

您可以檢視現有的自訂或NetApp提供的應用程式預設執行勾點。

步驟

1. 移至*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。

   您可以在結果清單中檢視所有已啟用或已停用的執行掛勾。您可以查看Hook的狀態、來源、以及何時執行（快照前或快照後）。若要檢視執行掛起的相關事件記錄、請前往左側導覽區域的*活動*頁面。

建立自訂執行掛勾

您可以為應用程式建立自訂執行掛勾。請參閱 "執行攔截範例" 如需攔截範例、您需要擁有擁有擁有者、管理員或成員權限、才能建立執行掛勾。

> 當您建立自訂Shell指令碼作為執行掛勾時、請記得在檔案開頭指定適當的Shell、除非您執行Linux命令或提供執行檔的完整路徑。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*新增連結*。
4. 在* HookDetails（掛機詳細資料）區域中、視掛機的執行時間而定、選擇*預先快照*或*快照後。
5. 輸入掛機的唯一名稱。
6. （選用）輸入執行期間要傳遞至掛機的任何引數、並在您輸入的每個引數之後按Enter鍵以記錄每個引數。
7. 在「* Container images"（* Container映像*）區域中、如果掛勾應針對應用程式中包含的所有容器映像執行、請啟用「* Apply to all Container images"（套用至所有容器映像）核取方塊。如果掛機只能對一個或多個指定的容器映像起作用、請在「要比對的容器映像名稱」欄位中輸入容器映像名稱。
8. 在*指令碼*區域中、執行下列其中一項：
   - 上傳自訂指令碼。
     i. 選取*上傳檔案*選項。
     ii. 瀏覽至檔案並上傳。
     iii. 為指令碼指定唯一名稱。
     iv. （選用）輸入其他系統管理員應該知道的任何指令碼附註。
   - 從剪貼簿貼入自訂指令碼。
     i. 選擇*從剪貼簿貼上*選項。

ii. 選取文字欄位、然後將指令碼文字貼到欄位中。

iii. 為指令碼指定唯一名稱。

iv. （選用）輸入其他系統管理員應該知道的任何指令碼附註。

9. 選取*新增攔截*。

停用執行掛勾

如果您想要暫時避免在應用程式快照之前或之後執行、可以停用執行掛勾。您需要擁有擁有擁有者、管理員或成員權限、才能停用執行掛勾。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*「動作*」下拉式清單以顯示您要停用的掛勾。
4. 選擇*停用*。

刪除執行掛勾

如果不再需要執行掛勾、您可以完全移除該掛勾。您需要擁有擁有擁有者、管理員或成員權限、才能刪除執行掛勾。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*「動作」*下拉式選單以選取您要刪除的掛勾。
4. 選擇*刪除*。

執行攔截範例

請使用下列範例、瞭解如何建立執行掛勾的架構。您可以使用這些hooks做為範本、或做為測試指令碼。

簡單的成功範例

這是一個簡單的勾點、可成功將訊息寫入標準輸出和標準錯誤。

```
#!/bin/sh

# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

簡單的成功範例（**Bash**版本）

這是一個簡單的攔截、成功將訊息寫入標準輸出和標準錯誤（寫入Bash）。

```
#!/bin/bash
```

```
# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
info "exit 0"
```

```
exit 0
```

簡單的成功範例（**zsh**版本）

這是一個簡單的勾點範例、可成功將訊息寫入標準輸出和標準錯誤、並寫入Z Shell。

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
# main
```

```
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

成功的引數範例

下列範例示範如何在攔截中使用args。

```
#!/bin/sh

# success_sample_args.sh
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
```

```
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

快照前/快照後掛機範例

以下範例說明如何將相同的指令碼同時用於快照前和快照後掛勾。

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]

# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}

#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}
```

```
#
# main
#

# check arg
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi

# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

故障範例

下列範例示範如何處理攔截式故障。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
```

```
#
# args: [the exit code to return]
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
```

```
exit ${argexitcode}
```

詳細故障範例

下列範例示範如何以更詳細的記錄功能來處理掛機中的失敗。

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
```

```
#

# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
done

error "exiting with error code 8"
exit 8
```

退出程式碼範例失敗

下列範例示範攔截失敗、並顯示結束程式碼。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
```

```
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

失敗後成功範例

下列範例示範第一次執行時發生掛機故障、但在第二次執行後成功。

```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
# args: None
```

```
#

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
    info "File does not exist.  Creating /tmp/hook-test.junk"
```

```
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
    exit 5
fi
```

檢視應用程式與運算健全狀況

檢視應用程式與叢集健全狀況的摘要

按一下*儀表板*即可查看應用程式、叢集及其健全狀況的高層級檢視。

應用程式方塊可協助您識別下列項目：

- 您目前管理的應用程式數量。
- 這些託管應用程式是否健全。
- 應用程式是否受到完整保護（如果有最近的備份可用、則會受到保護）。
- 已探索但尚未管理的應用程式數量。

  理想情況下、這個數字會是零、因為您會在發現應用程式之後管理或忽略這些應用程式。然後、您可以監控儀表板上探索到的應用程式數量、以識別開發人員何時將新應用程式新增至叢集。

請注意、這些不只是數字或狀態、您可以從每個位置深入瞭解。例如、如果應用程式未受到完整保護、您可以將游標停留在圖示上、以識別哪些應用程式未受到完整保護、這也是原因之一。

叢集區塊提供類似叢集健全狀況的詳細資料、您可以深入瞭解更多詳細資料、就像使用應用程式一樣。

檢視叢集的健全狀況和詳細資料

將Kubernetes叢集新增至Astra Control之後、即可檢視叢集的詳細資料、例如其位置、工作節點、持續磁碟區和儲存類別。

步驟

1. 在Astra Control Service UI中、選取* Clusters*。
2. 在「叢集」頁面上、選取您要檢視其詳細資料的叢集。

   > 如果叢集處於「移除」狀態、但叢集和網路連線狀況良好（外部使用Kubernetes API存取叢集的嘗試成功）、則您提供給Astra Control的Kubeconfig可能不再有效。這可能是因為叢集上的憑證輪替或過期。若要修正此問題、請使用更新Astra Control中與叢集相關的認證資料 "Astra Control API"。

3. 查看*概述*、*儲存設備*和*活動*索引標籤上的資訊、以尋找您要尋找的資訊。
   - 總覽：工作節點的詳細資料、包括其狀態。
   - * Storage *：與運算相關的持續磁碟區、包括儲存類別和狀態。
   - 活動：與叢集相關的活動。

您也可以從Astra Control Service * Dashboard 開始檢視叢集資訊。在*叢集*索引標籤的*資源摘要*下、您可以選取受管理的叢集、然後前往*叢集*頁面。進入「*叢集」頁面之後、請依照上述步驟操作。

檢視應用程式的健全狀況和詳細資料

在您開始管理應用程式之後、Astra Control會提供應用程式的詳細資料、讓您識別應用程式的狀態（是否健全）、保護狀態（是否在故障時受到完整保護）、Pod、持續儲存設備等。

步驟

1. 選取*應用程式*、然後選取應用程式名稱。
2. 選取「四處」以尋找您要尋找的資訊：

應用程式狀態

提供反映Kubernetes應用程式狀態的狀態。例如、Pod和持續磁碟區是否在線上？如果某個應用程式不健全、您必須查看Kubernetes記錄檔、在叢集上進行疑難排解。Astra Control並未提供資訊來協助您修正毀損的應用程式。

應用程式保護狀態

提供應用程式受到保護的程度狀態：

- 完全保護：應用程式有作用中的備份排程、而且備份成功的時間不到一週
- 部分保護：應用程式有作用中的備份排程、作用中的快照排程、或成功的備份或快照
- 未受保護：未受到完整保護或部分保護的應用程式。

  您必須等到最近進行備份之後、才能獲得完整保護。這很重要、因為備份儲存在遠離持續磁碟區的物件存放區中。如果故障或意外將叢集及其持續儲存設備擦除、則需要備份才能恢復。快照無法讓您恢復。

總覽

與應用程式相關聯的Pod狀態資訊。

資料保護

可讓您設定資料保護原則、並檢視現有的快照與備份。

儲存設備

顯示應用程式層級的持續磁碟區。持續磁碟區的狀態是從Kubernetes叢集的觀點來看。

資源

可讓您驗證要備份和管理的資源。

活動

Astra Control與應用程式相關的活動。

管理儲存庫

透過新增其他儲存區、以及變更雲端供應商Kubernetes叢集的預設儲存區、來管理Astra用於備份與複製的儲存區。

只有管理員可以新增及修改儲存區。

Astra Control如何使用鏟斗

當您開始管理第一個Kubernetes叢集時、Astra Control Service會在與受管理叢集相同的地理區域中、為您的雲端供應商建立預設儲存區。

Astra Control Service使用此預設儲存區來進行您所建立的備份與複製。然後、您可以使用備份來還原及複製叢集之間的應用程式。

如果您在Astra Control Service中新增其他儲存庫、則可在建立保護原則時從這些儲存庫中選取。您也可以變更Astra Control Service用於臨機操作備份與複製的預設儲存區。您無法將儲存庫從一個雲端供應商複製到另一個雲端供應商。若要將資料從一個雲端供應商移轉至另一個雲端供應商、請改為複製相關的應用程式。

Astra Control Service會在開始備份或實體複本之前、先檢查目標儲存區是否可供存取。

檢視現有的貯體

檢視Astra Control Service可用的庫存箱清單、以判斷其狀態、並識別雲端供應商的預設庫存箱。

桶可能具有下列任一狀態：

搁置中
: 新增儲存區之後、它會從擱置狀態開始、而Astra Control則會第一次查看它。

可用
: 此鏟斗可供Astra Control使用。

已移除
: 鏟斗目前無法運作。將滑鼠游標暫留在狀態圖示上、以找出問題所在。

  如果某個儲存區處於「已移除」狀態、您仍可將其設為預設儲存區、並將其指派給保護排程。但如果儲存區在資料保護作業開始時未處於可用狀態、則該作業將會失敗。

步驟

1. 在「管理您的儲存設備」下、選取「桶」。

   顯示Astra Control Service可用的貯體清單。

新增額外的儲存庫

在雲端供應商開始管理叢集之後、您可以隨時新增其他資源庫。這可讓您在建立保護原則時、在儲存區之間進行選擇、並變更臨機操作備份與複製的預設儲存區。

請注意、Astra Control Service無法讓您在新增後移除庫位。

您需要的是 **#8217** ；需要的是什麼

- 雲端供應商中現有儲存庫的名稱。
- 如果您的儲存庫位於Azure、則必須屬於名為_Astra備份rg-_的資源群組。

步驟

1. 在「管理您的儲存設備」下、選取「桶」。
2. 選擇*「Add」（新增）*、然後依照提示新增儲存區。
    - 類型：選擇您的雲端供應商。

      您的雲端供應商只有在Astra Control Service開始管理在該雲端供應商中執行的叢集之後、才能使用。

    - 現有庫位名稱：輸入庫位名稱。
    - 說明：選擇性地輸入桶的說明。
    - 將此儲存區設為此雲端的預設儲存區：選擇是否要使用此儲存區做為臨機操作備份與複製的預設儲存區。
    - 選取認證資料：選擇提供Astra Control Service所需權限的認證資料、以管理儲存庫。
3. 選取*「Add*（新增*）」以新增儲存區。

Astra Control Service新增額外的儲存庫。您現在可以在建立保護原則時選擇儲存庫。

變更預設儲存區

變更Astra Control Service用於備份與複製的預設儲存區。每個雲端供應商都有自己的預設儲存庫。

Astra Control Service使用雲端供應商的預設儲存庫進行臨機操作備份、如果您不選擇從現有備份複製、則可用於臨機操作複製。

步驟

1. 在「管理您的儲存設備」下、選取「桶」。
2. 在* Actions （動作）欄中選取您要編輯的儲存區的下拉式清單。
3. 選取*將此儲存區設為此雲端的預設儲存區*。
4. 選擇*更新*。

移除貯體

您可以移除不再使用或不健全的庫位。您可能會想要這麼做、讓物件存放區組態保持簡單且最新狀態。

您無法移除預設的儲存區。如果您要移除該儲存區、請先選取另一個儲存區做為預設值。

您需要的是 **#8217** ；需要的是什麼

- 開始之前、您應檢查以確保此儲存區沒有執行中或已完成的備份。
- 您應該檢查以確保儲存庫不會用於任何排程的備份。

如果有、您將無法繼續。

步驟

1. 從左側導覽中選取*鏟斗*。
2. 從* Actions（操作）功能表中、選取*移除。

> Astra Control會先確保不會有使用儲存庫進行備份的排程原則、而且您要移除的儲存庫中沒有作用中的備份。

3. 輸入「移除」以確認動作。
4. 選擇*是、移除桶*。

如需詳細資訊、請參閱

- "使用Astra Control API"

管理您的帳戶

設定帳單

Astra Control的免費方案可讓您管理帳戶中最多10個應用程式。如果您想要管理10個以上的應用程式、則必須將「免費方案」升級為「優質方案」、以設定帳單。

帳單總覽

使用Astra Control Service的相關成本有兩種：NetApp對Astra Control Service的收費、以及您的雲端供應商對持續磁碟區和物件儲存的收費。

Astra Control服務計費

Astra Control Service提供三項計畫：

免費方案
: 免費管理最多10個應用程式。

優質**PayGo**
: 管理無限數量的應用程式、每個應用程式每分鐘收費$.005。

優質訂閱
: 以折扣價預先付款、每年訂閱一次、可讓您管理每個應用程式套件最多10個應用程式。請聯絡NetApp銷售人員、視組織需求購買任意數量的套件、例如購買3個套件、即可從Astra Control Service管理30個應用程式。如果您管理的應用程式數量超過年度訂閱所允許的數量、則每個應用程式的超額使用率將高達每分鐘$0.005（與Premium PayGo相同）。

  如果您還沒有Astra Control帳戶、購買Premium訂閱會自動為您建立Astra Control帳戶。如果您有現有的免費方案、系統會自動將您轉換為「優質訂閱」。

當您建立Astra Control帳戶時、系統會自動為您註冊免費方案。Astra Control的儀表板顯示您目前在10個免費應

用程式中管理的應用程式數量。

如果您嘗試管理第11個應用程式、Astra Control會通知您已達到「免費方案」的上限。接著會提示您從「免費方案」升級至「優質方案」。

您可以隨時升級至優質方案。升級後、Astra Control會開始向您收取帳戶中受管理的_all_應用程式的費用。前10個應用程式不會留在「免費方案」中。

Google Cloud帳單

當您使用Astra Control Service管理GKE叢集時、NetApp Cloud Volumes Service 支援持續的磁碟區、而應用程式的備份則儲存在Google Cloud Storage儲存庫中。

- "檢視Cloud Volumes Service 報價詳細資料以供參考"。

  請注意、Astra Control Service支援所有服務類型和服務層級。您使用的服務類型取決於 "Google Cloud區域"。

- "檢視Google Cloud儲存桶的定價詳細資料"。

Microsoft Azure帳單

當您使用Astra Control Service管理高效能叢集時、持續的磁碟區會以Azure NetApp Files 支援功能為後盾、而應用程式的備份則會儲存在Azure Blob容器中。

- "檢視Azure NetApp Files 報價詳細資料以供參考"。
- "檢視Microsoft Azure Blob儲存設備的定價詳細資料"。

重要注意事項

- 您的計費計畫依Astra Control帳戶計算。

  如果您有多個帳戶、則每個帳戶都有自己的計費計畫。

- 您的Astra Control費用包含管理Kubernetes應用程式的費用。您需要由雲端供應商另行付費、以取得持續磁碟區的儲存後端。

  "深入瞭解Astra Control定價"。

- 每個計費期間都會在每月的最後一天結束。
- 您無法從優質方案降級為免費方案。

從「免費方案」升級至「優質**PayGo**方案」

隨時升級您的計費計畫、即可開始使用Astra Control管理10多個應用程式、而且您可以隨需付費。您只需要一張有效的信用卡。

步驟

1. 選擇* Account（帳戶）、然後選擇 Billing（計費）*。
2. 在「方案」下、前往* Premium PayGo*並選取「立即升級」。

3. 請提供有效信用卡的付款詳細資料、然後選擇*升級至Premium Plan *。

> 若信用卡即將到期、Astra Control將會寄送電子郵件給您。

您現在可以管理10多個應用程式。Astra Control開始為您目前管理的_all_應用程式充電。

從「免費方案」升級至「優質訂閱」

請聯絡NetApp銷售人員、以每年訂閱的折扣率預付費用。

步驟

1. 選擇* Account（帳戶）、然後選擇 Billing（計費）*。
2. 在「方案」下、移至* Premium Subscription*並選取*聯絡銷售*。
3. 請提供詳細資料給銷售團隊、以開始流程。

NetApp銷售代表將與您聯絡、以處理您的訂單。訂單完成後、Astra Control會在「帳單」索引標籤上反映您目前的計畫。

檢視您目前的成本和帳單記錄

Astra Control會顯示您目前的每月成本、以及應用程式的詳細帳單記錄。

步驟

1. 選擇* Account（帳戶）、然後選擇 Billing（計費）*。

   您目前的成本會顯示在帳單總覽下方。

2. 若要依應用程式檢視帳單記錄、請選取*帳單記錄*。

   Astra Control會顯示每個應用程式的使用時間和成本。使用分鐘數是指在計費期間、Astra Control管理您應用程式的分鐘數。

3. 選取下拉式清單以選取上個月。

變更**Premium PayGo**的信用卡

如有需要、您可以變更Astra Control檔案中的信用卡進行收費。

步驟

1. 選擇*帳戶>帳單>付款方式*。
2. 選取「組態」圖示。
3. 修改信用卡。

邀請及移除使用者

邀請使用者加入您的Astra Control帳戶、並移除不應再擁有帳戶存取權的使用者。

邀請使用者

帳戶擁有者和管理員可以邀請其他使用者加入Astra Control帳戶。

步驟

1. 請確定使用者擁有 "Cloud Central登入"。
2. 選擇*帳戶*。
3. 在*使用者*索引標籤中、選取*邀請*。
4. 輸入使用者名稱、電子郵件地址及其角色。

   請注意下列事項：

   - 電子郵件地址必須與使用者用來註冊Cloud Central的電子郵件地址相符。
   - 每個角色都提供下列權限：
     - *擁有者*具有管理權限、可刪除帳戶。
     - *管理*具有成員權限、可以邀請其他使用者。
     - *成員*可以完全管理應用程式和叢集。
     - *檢視器*可以檢視資源。
5. 若要新增限制給具有成員或檢視者角色的使用者、請啟用*限制角色限制*核取方塊。

   如需新增限制的詳細資訊、請參閱 "管理角色"。
6. 若要邀請其他使用者、請選取*新增其他使用者*、然後輸入新使用者的資訊。

   您一次最多可邀請10位使用者。您可以在「邀請使用者」對話方塊的左側、在您要邀請的使用者之間瀏覽。
7. 選擇*邀請使用者*。

使用者將會收到一封電子郵件、邀請他們加入您的帳戶。

變更使用者的角色

帳戶擁有者可以變更所有使用者的角色、而帳戶管理員可以變更具有管理員、成員或檢視者角色的使用者角色。

步驟

1. 選擇*帳戶*。
2. 在「使用者」索引標籤中、選取使用者「動作」欄中的功能表。
3. 選擇*編輯角色*。
4. 選取新角色。
5. 若要新增限制給具有成員或檢視者角色的使用者、請啟用*限制角色限制*核取方塊。

   如需新增限制的詳細資訊、請參閱 "管理角色"。
6. 選擇* Confirm（確認）*。

Astra Control會根據您選取的新角色來更新使用者權限。

移除使用者

擁有者角色的使用者可以隨時從帳戶中移除其他使用者。

步驟

1. 選擇*帳戶*。
2. 在*使用者*索引標籤中、選取您要移除的使用者。
3. 在「動作」欄中選取功能表、然後選取「移除使用者」。
4. 出現提示時、請輸入「移除」確認刪除、然後選擇*是、移除使用者*。

Astra Control會將使用者從帳戶中移除。

管理角色

您可以新增命名空間限制、並將使用者角色限制在這些限制中、藉此管理角色。這可讓您控制組織內資源的存取。您可以使用Astra Control UI或 "Astra Control API" 以管理角色。

將命名空間限制新增至角色

管理員或擁有者使用者可以新增命名空間限制。

步驟

1. 在*管理您的帳戶*導覽區域中、選取*帳戶*。
2. 選取*使用者*索引標籤。
3. 在「動作」欄中、選取具有「成員」或「檢視者」角色的使用者功能表按鈕。
4. 選擇*編輯角色*。
5. 啟用「限制角色*」核取方塊。

   此核取方塊僅適用於「成員」或「檢視者」角色。您可以從*角色*下拉式清單中選取不同的角色。

6. 選取*新增限制*。

   您可以依命名空間或命名空間標籤檢視可用限制清單。

7. 在*限制類型*下拉式清單中、視命名空間的設定方式而定、選取* Kubernetes命名空間*或* Kubernetes命名空間標籤*。
8. 從清單中選取一或多個命名空間或標籤、以構成限制、限制角色只能使用這些命名空間。
9. 選擇* Confirm（確認）*。

   「編輯角色」頁面會顯示您為此角色選擇的限制清單。

10. 選擇* Confirm（確認）*。

    在「帳戶」頁面上、您可以在「角色」欄中檢視任何成員或檢視者角色的限制條件。

> 如果您啟用角色的限制、並選取* Confirm（確認）*而不新增任何限制、則該角色會被視為具有完整限制（該角色無法存取指派給命名空間的任何資源）。

從角色移除命名空間限制

管理員或擁有者使用者可以從角色移除命名空間限制。

步驟

1. 在*管理您的帳戶*導覽區域中、選取*帳戶*。
2. 選取*使用者*索引標籤。
3. 在「動作」欄中、選取具有作用中限制之「成員」或「檢視者」角色的使用者功能表按鈕。
4. 選擇*編輯角色*。

   「編輯角色」對話方塊會顯示角色的作用中限制。

5. 選取您需要移除之限制右側的* X*。
6. 選擇* Confirm（確認）*。

以取得更多資訊

- "使用者角色和命名空間"

新增及移除認證資料

隨時從您的帳戶新增及移除雲端供應商認證資料。Astra Control會使用這些認證資料來探索Kubernetes叢集、叢集上的應用程式、並代表您配置資源。

請注意、Astra Control中的所有使用者都共用相同的認證集。

新增認證資料

將認證新增至Astra Control的最常見方法是管理叢集、但您也可以從「帳戶」頁面新增認證。然後、當您管理額外的Kubernetes叢集時、便可選擇認證資料。

您需要的是 **#8217** ；需要的是什麼

- 若為GKE、您應該擁有具有所需權限之服務帳戶的服務帳戶金鑰檔案。 "瞭解如何設定服務帳戶"。
- 若為高效能、您應該擁有Json檔案、其中應包含建立服務主體時Azure CLI的輸出。 "瞭解如何設定服務主體"。

  如果您未將Azure訂閱ID新增至Json檔案、您也需要Azure訂閱ID。

步驟

1. 選擇*帳戶>認證*。
2. 選取*新增認證*。
3. 選擇* Microsoft Azure 或 Google Cloud Platform *。

4. 在Astra Control中輸入認證資料的名稱、以區分這些認證與其他認證資料。
5. 提供必要的認證資料。
   a. * Microsoft Azure *：上傳Json檔案或從剪貼簿貼上Json檔案的內容、以提供Astra Control有關Azure服務主體的詳細資料。

      當您建立服務主體時、Json檔案應包含Azure CLI的輸出。它也可以包含您的訂閱ID、以便自動新增至Astra Control。否則、您必須在提供Json之後手動輸入ID。

   b. * Google Cloud Platform *：上傳檔案或從剪貼簿貼上內容、以提供Google Cloud服務帳戶金鑰檔案。
6. 選取*新增認證*。

您現在可以在將叢集新增至Astra Control時、選取認證資料。

移除認證資料

隨時從帳戶移除認證資料。您只能在之後移除認證 "取消管理所有叢集"、除非您正在旋轉認證資料（請參閱 [Rotate credentials]）。

> 您新增至Astra Control的第一組認證資料一律使用中、因為Astra Control使用認證資料來驗證備份儲存區。最好不要移除這些認證資料。

步驟

1. 選擇*帳戶>認證*。
2. 選取您要移除之認證資料的「狀態」欄中的下拉式清單。
3. 選擇*移除*。
4. 輸入要確認刪除的認證資料名稱、然後選取*是、移除認證*。

Astra Control會從帳戶移除認證資料。

旋轉認證資料

您可以在帳戶中旋轉認證資料。如果您旋轉認證資料、請在維護期間（排程或隨需）無備份進行時、於維護期間旋轉認證資料。

步驟

1. 依照中的步驟移除現有認證 [Remove credentials]。
2. 依照中的步驟新增認證資料 [Add credentials]。
3. 更新所有儲存庫以使用新認證：
   a. 在「管理您的儲存設備」下、選取「桶」。
   b. 在* Actions （動作）欄中選取您要編輯的儲存區的下拉式清單。
   c. 選擇*編輯*。
   d. 在「選取認證資料」區段中、選擇您新增至Astra Control的新認證資料。
   e. 選擇*更新*。
   f. 對系統上的任何剩餘儲存區重複步驟* b*到* e*。

Astra Control開始使用新的雲端供應商認證資料。

監控帳戶活動

您可以檢視Astra Control帳戶中活動的詳細資料。例如、當邀請新使用者、新增叢集或擷取快照時。您也可以將帳戶活動匯出至CSV檔案。

檢視**Astra Control**中的所有帳戶活動

1. 選擇*活動*。
2. 使用篩選器縮小活動清單範圍、或使用搜尋方塊找到您想要的確切內容。
3. 選取*匯出至CSV*、將您的帳戶活動下載至CSV檔案。

檢視特定應用程式的帳戶活動

1. 選取*應用程式*、然後選取應用程式名稱。
2. 選擇*活動*。

檢視叢集的帳戶活動

1. 選取*叢集*、然後選取叢集名稱。
2. 選擇*活動*。

檢視事件的可能行動

1. 選擇*活動*。
2. 選取需要注意的事件。
3. 選取*「採取行動」*下拉式選項。

   您可在此清單中檢視可能採取的修正行動、檢視與問題相關的文件、並取得協助解決問題的支援。

檢視及管理通知

Astra Control會在行動完成或失敗時通知您。例如、如果成功完成應用程式的備份、您會看到通知。

介面右上角有未讀取通知的數量。

您可以檢視這些通知並將其標示為已讀取（如果您想要像我們一樣清除未讀取的通知、就很方便）。

步驟

1. 選取右上角的未讀取通知數。
2. 檢閱通知、然後選取*標示為已讀取*或*顯示所有通知*。

   如果您選取*顯示所有通知*、則會載入「通知」頁面。

3. 在*通知*頁面上、檢視通知、選取您要標示為已讀的通知、選取*行動*、然後選取*標示為已讀*。

關閉您的帳戶

如果您不再需要Astra Control帳戶、您可以隨時關閉帳戶。

> 當您關閉帳戶時、Astra Control自動建立的庫位會自動刪除。

步驟

1. "取消管理所有應用程式和叢集"。
2. "從Astra Control移除認證資料"。
3. 選擇*帳戶>帳單>付款方式*。
4. 選擇*關閉帳戶*。
5. 輸入您的帳戶名稱、然後確認關閉帳戶。

取消管理應用程式和叢集

從Astra Control移除不再需要管理的任何應用程式或叢集。

停止管理應用程式

停止管理不再想從Astra Control備份、快照或複製的應用程式。

- 任何現有的備份與快照都會刪除。
- 應用程式與資料仍可繼續使用。

步驟

1. 選擇*應用程式*。
2. 選取您不想再管理的應用程式核取方塊。
3. 選取* Action"下拉式清單、然後選取* UnManage application/s*。
4. 確認您要取消管理應用程式、然後選取*是、取消管理應用程式*。

Astra Control停止管理應用程式。

停止管理叢集

停止管理不再想從Astra Control管理的叢集。最佳實務做法是、建議您在透過GCP刪除叢集之前、先從Astra Control中移除叢集。

- 此動作會使叢集無法由Astra Control管理。它不會對叢集的組態進行任何變更、也不會刪除叢集。
- Astra Trident不會從叢集解除安裝。 "瞭解如何解除安裝Astra Trident"。

步驟

1. 選擇*叢集*。
2. 選取您不想再管理之叢集的核取方塊。

3. 選取「動作」下拉式清單、然後選取「取消管理」。
4. 確認您要取消管理叢集、然後選取*是、取消管理叢集*。

Astra Control停止管理叢集。

從雲端供應商刪除叢集

在刪除Kubernetes叢集之前、如果叢集的NetApp儲存類別中含有持續磁碟區（PV）、您必須先依照下列其中一種方法刪除持續磁碟區宣告（PVc）。刪除叢集之前刪除PVC和PV、可確保您不會收到雲端供應商的非預期帳單。

- 方法**#1**：從叢集刪除應用程式工作負載命名空間。請勿刪除Trident命名空間。
- 方法**#2**：刪除PVCS和Pod、或是掛載PV的部署。

當您從Astra Control管理Kubernetes叢集時、該叢集上的應用程式會使用Cloud Volumes Service 原地不動或Azure NetApp Files 用作持續磁碟區的儲存後端。如果您從雲端供應商刪除叢集、但未先移除PV、則會隨叢集一起刪除後端磁碟區。

使用上述方法之一、將會從叢集刪除對應的PV。刪除叢集之前、請先確定叢集上的NetApp儲存類別上沒有任何PV。

如果您在刪除叢集之前未刪除持續的磁碟區、則需要從Cloud Volumes Service 適用於Google Cloud或Azure NetApp Files 從停止功能中手動刪除後端磁碟區。

版權資訊

Copyright©2022 NetApp、Inc.版權所有。美國印製本文件中版權所涵蓋的任何部分、不得以任何形式或任何方式（包括影印、錄製、 在未事先取得版權擁有者書面許可的情況下、在電子擷取系統中進行錄音或儲存。

衍生自受版權保護之NetApp資料的軟體必須遵守下列授權與免責聲明：

本軟體係由NetApp「依現狀」提供、不含任何明示或暗示的保證、包括但不限於適售性及特定用途適用性的暗示保證、特此聲明。在任何情況下、NetApp均不對任何直接、間接、偶發、特殊、示範、 或衍生性損害（包括但不限於採購替代商品或服務；使用損失、資料或利潤損失； 或業務中斷）、無論是在合約、嚴格責任或侵權行為（包括疏忽或其他）中、無論是因使用本軟體而產生的任何責任理論（包括疏忽或其他）、即使已被告知可能造成此類損害。

NetApp保留隨時變更本文所述之任何產品的權利、恕不另行通知。除非NetApp以書面明確同意、否則NetApp不承擔因使用本文所述產品而產生的任何責任或責任。使用或購買本產品並不代表NetApp擁有任何專利權利、商標權利或任何其他智慧財產權。

本手冊所述產品可能受到一或多個美國國家/地區的保護專利、國外專利或申請中。

限制權利圖例：政府使用、複製或揭露受DFARS 252.277-7103（1988年10月）和FAR 52-227-19（1987年6月）技術資料與電腦軟體權利條款（c）（1）（ii）分段所述限制。

商標資訊

NetApp、NetApp標誌及所列的標章 http://www.netapp.com/TM 為NetApp、Inc.的商標。其他公司和產品名稱可能為其各自所有者的商標。